

Implementation of AES-128 Algorithm for Secure UART Communication on Artix-7 FPGA

A REPORT ON PROJECT OF Guided Research Reading (4EL05)

Bachelor of Technology
(Electronics Engineering)

Submitted by:

ROHIT VIJAY GUPTA
(21EL044)

Under the guidance of:

Prof. Anita Bhatt



Birla Vishvakarma Mahavidyalaya
Engineering Collage (An Autonomous Institution)

Vallabh Vidyanagar - 388 120

Affiliated to Gujarat Technological University

Certificate

This is to certify that the Guided Research Reading entitled “**Implementation of AES-128 Algorithm for Secure UART Communication on Artix-7 FPGA**” has been carried out by **Rohit Gupta** in the **7th semester** under my guidance in Electronics Engineering, **Birla Vishvakarma Mahavidyalaya**, Vallabh Vidyanagar, during the academic year 2024-25.

Guided by: **Prof. Anita Bhatt**

Head of the Department

Acknowledgement

It is with immense pleasure and a sense of pride that we present this mini-project entitled **“Implementation of AES-128 Algorithm for Secure UART Communication on Artix-7 FPGA.”** This project has been a rewarding journey, combining academic learning with practical application, and it stands as a testament to our dedication and commitment to advancing secure communication technologies.

We would like to extend our heartfelt gratitude to **Prof. Anita Bhatt** for her invaluable guidance and constant encouragement throughout the course of this project. Her expert insights, constructive feedback, and unwavering support have been instrumental in shaping this work.

We are equally grateful to **Dr. T. D. Pawar**, Head of the Electronics Department at Birla Vishvakarma Mahavidyalaya, Vallabh Vidyanagar, Anand, for inspiring us to undertake this challenging project. His leadership and continuous motivation created an environment conducive to innovation and excellence.

We would also like to express our sincere appreciation to **Birla Vishvakarma Mahavidyalaya (BVM)** for providing the necessary resources and project components, which played a vital role in the successful implementation of this project. The facilities and support offered by the college have greatly enriched our learning experience and enabled us to bring this project to fruition.

Their support, combined with the resources and facilities provided by the department, has greatly enriched our understanding and enabled us to successfully execute this project. We deeply appreciate their confidence in our abilities and their readiness to assist us whenever required.

This project represents the culmination of our efforts, and we hope it serves as a small yet significant contribution to the field of secure communication systems. We are sincerely thankful for the opportunity to explore and innovate under their guidance.

Abstract

This project aims to significantly enhance the security of UART (Universal Asynchronous Receiver/Transmitter) communication by integrating AES (Advanced Encryption Standard) encryption. UART is a widely adopted serial communication protocol used in embedded systems for transmitting data between microcontrollers and peripherals. However, the traditional UART protocol lacks any built-in security mechanisms, making it vulnerable to unauthorized data access or tampering. This limitation is critical in applications where sensitive data is transmitted, such as in industrial systems, IoT devices, or remote-controlled systems.

To address this security gap, the project incorporates AES encryption, a symmetric-key encryption algorithm known for its robustness and efficiency. AES is widely used in securing communications due to its proven security and relatively low computational requirements, making it well-suited for hardware implementations. By encrypting data before transmission and decrypting it upon reception, AES ensures that the data remains confidential and protected from interception or unauthorized access.

The project involves the design and implementation of AES encryption and decryption on FPGA hardware, specifically utilizing the Artix-7 Basys 3 FPGA board. The FPGA's parallel processing capabilities enable high-speed encryption and decryption, ensuring minimal latency in secure communication. On the transmitting side, the plaintext data is encrypted using a 128-bit AES key, and the encrypted data is sent over UART. On the receiving side, the data is decrypted using the same AES key to restore the original message.

This integration of AES encryption with UART communication effectively combines the simplicity and wide compatibility of UART with the advanced security features of AES, offering a reliable and secure solution for embedded systems and IoT applications. The project also demonstrates how hardware-based AES encryption can be seamlessly implemented in a practical environment, ensuring both the security and reliability of data transfer in a variety of real-world applications where sensitive information needs to be transmitted securely.

Keyword : AES, UART, Encryption, Decryption, FPGA, BASYS3

Table of Contents

Certificate	i
Acknowledgement	ii
Abstract	iii
List of Figures	v
Chapter 1: Introduction.....	1
1.1 Literature Review	2
1.2 Motivation for the Project.....	3
1.3 Work Flow.....	4
Chapter 2: UART	7
2.1 How UART Works.....	8
2.2 Frame Format	8
2.2.1 Start Bit.....	9
2.2.2 Data bit	9
2.2.3 Parity bit.....	10
2.2.4 Stop bit	10
2.3 Challenges in securing Uart.....	11
2.3.1 Data Interception	11
2.3.2 Lack of Authentication.....	11
2.3.3 Physical Access.....	11
2.3.4 Data Integrity	11
2.3.5 Insecure Default Settings	11
Chapter 3: AES Algorithm	12
3.1 Introduction to AES	12
3.1.1 AES Key Sizes and Encryption Rounds	12
3.1.2 AES Block Structure	12
3.1.3 Significance of the State Matrix	13
3.1.4 Key Expansion Process	13
3.2 AES Encryption Process	14
3.2.1 Add RoundKey	14
3.2.2 SubBytes	15
3.2.3 ShiftRows	16
3.2.4 MixColumns	16
3.2.5 Flow Chart of Encryption.....	17

3.3 AES Decryption Process	18
3.3.1 ADD Round Key	18
3.3.2 Inverse ShiftRows	19
3.3.3 Inverse SubBytes	20
3.3.4 Inverse MixColumns	21
3.3.5 Flow Chart of Decryption.....	21
3.4 AES Encryption Example.....	22
Chapter 4: Integration OF AES in UART	24
4.1 Block Diagram Explanation	24
4.2 Data Flow.....	25
Chapter 5: Software Simulation	26
5.1 Testbench Description	26
5.2 Simulation Workflow	26
5.3 Results	27
5.4 RTL Schematic Description.....	28
Chapter 6: HardWare Implementation	29
6.1 Basys 3 FPGA Board.....	29
6.2 Data Flow and Encryption/Decryption Process	31
6.3 Hardware Setup	33
6.3.1 PC to Basys 3 FPGA (Board 1)	33
6.3.2 Basys 3 FPGA (Board 1) to Basys 3 FPGA (Board 2)	33
6.3.3 Basys 3 FPGA (Board 1) to Arduino	33
6.3.4 Basys 3 FPGA (Board 2) to PC	34
6.4 RTL Schematic.....	34
Chapter 7: Result	35
7.1 Safety Analysis and Improvements.....	35
7.1.1 Importance of Safety Analysis	35
7.1.2 Brute Force Attack	35
7.1.3 Square Attack	36
Chapter 8: Conclusion	37
References	37

List of Figures

Figure 1.1:	Work Flow	4
Figure 2.1:	UART Serial Communication.....	8
Figure 2.2:	UART Frame Format.....	8
Figure 2.3:	UART Start bit.....	9
Figure 2.4:	UART Data bit.....	9
Figure 2.5:	UART Parity bit	10
Figure 2.6:	UART Stop bit.....	10
Figure 3.1:	AES state Matrix	13
Figure 3.2:	AES Key	14
Figure 3.3:	ADDRound Key.....	14
Figure 3.4:	S box	15
Figure 3.5:	SubBytes	15
Figure 3.6:	ShiftRows	16
Figure 3.7:	MixColumns	17
Figure 3.8:	Encryption Flowchart	18
Figure 3.9:	ADD RoundKey.....	19
Figure 3.10:	Inverse ShiftRows.....	19
Figure 3.11:	Inverse S-box.....	20
Figure 3.12:	Inverse SybBytes.....	20
Figure 3.13:	Inverse MixColumns	21
Figure 3.14:	Decryption Flow Chart	22
Figure 3.15:	AES Example	23
Figure 4.1:	Integration OF AES in UART.....	24
Figure 5.1:	Vivado Console Output	27
Figure 5.2:	Vivado Simulation	27
Figure 5.3:	RTL Schematic	28
Figure 5.4:	Inside AES encryption Module.....	28
Figure 5.5:	Inside Uart Module	28
Figure 6.1:	Basys 3 Board.....	30
Figure 6.2:	Data Transmission Flowchart	32
Figure 6.3:	Hardware Setup	33
Figure 6.4:	Hardware Setup	34
Figure 7.1:	Estimation of Years to Break AES.....	36

Chapter 1: Introduction

In the modern era of rapidly evolving technology, security plays a crucial role in safeguarding sensitive data during transmission. One of the most widely used methods for communication between embedded systems is the *Universal Asynchronous Receiver-Transmitter (UART)* protocol. However, UART communication is inherently insecure, making it vulnerable to various types of cyber-attacks such as eavesdropping and data manipulation. To address these security concerns, encryption techniques are often implemented to ensure the confidentiality and integrity of the transmitted data.

The *Advanced Encryption Standard (AES)* is one of the most robust encryption algorithms and has been widely adopted across various applications due to its high security and efficiency. AES-128, which utilizes a 128-bit key for both encryption and decryption, is particularly popular for securing data in modern communication systems. Its security strength and efficiency make it ideal for hardware-based implementations, particularly in resource-constrained embedded systems.

In this project, we aim to implement the **AES-128 encryption algorithm** for secure UART communication using the **Artix-7 FPGA** platform. The Artix-7 FPGA, developed by Xilinx, offers an optimal combination of logic resources, low power consumption, and high speed, making it an ideal platform for implementing encryption algorithms in hardware. The flexibility of FPGA technology allows for parallel processing, which significantly enhances the efficiency of encryption and decryption tasks, making it suitable for real-time applications.

The main objective of this project is to design a hardware architecture that integrates AES-128 encryption with UART communication. The system will encrypt the data before transmission and decrypt it upon reception, ensuring that the data remains secure during transit. The encryption and decryption operations will be implemented in hardware on the Artix-7 FPGA, providing a secure communication channel with minimal latency.

This project will not only demonstrate how to secure UART communication but also highlight the benefits of hardware-based encryption. By using FPGA technology, we aim to achieve faster encryption and decryption speeds compared to software-based approaches. The project has practical applications in a wide range of embedded systems, including Internet of Things (IoT) devices, industrial automation systems, and autonomous vehicles, where secure communication is vital for maintaining data privacy and integrity.

Through this work, we will explore the challenges and solutions in implementing AES-128 encryption in hardware and examine the effectiveness of this solution in providing secure communication for embedded systems.

1.1 Literature Review

The Universal Asynchronous Receiver Transmitter (UART) protocol is a cornerstone of serial communication in embedded systems due to its simplicity and reliability. Huang and Sheng (2024) conducted an in-depth analysis of the UART communication protocol, addressing its operational challenges and its adaptability in diverse real-world applications. Their study emphasizes the importance of UART in scenarios requiring consistent and efficient data exchange, particularly under varying environmental and operational conditions. This foundational research provides crucial insights into ensuring robust communication for secure applications [1]. Furthermore, Kong et al. (2023) focused on the hardware implementation of UART using Verilog HDL, highlighting design optimizations that reduce resource utilization while maintaining reliable performance. Their findings serve as a practical guideline for integrating UART into FPGA systems, ensuring hardware efficiency and reliability in high-performance applications [2]. An et al. (2023) proposed a significant advancement with adaptive baud rate technology for UART systems, enabling dynamic adjustment of baud rates to enhance system flexibility and efficiency. This innovation is particularly useful in applications requiring real-time adaptability to changing communication conditions [3].

The Advanced Encryption Standard (AES) algorithm has emerged as a gold standard in cryptographic systems due to its unparalleled security and efficiency. Rameel and Asif (2024) provided a comparative analysis of AES against other cryptographic algorithms such as DES, 3DES, and RSA. Their study concludes that AES offers superior computational efficiency and resistance to cryptographic attacks, making it the preferred choice for secure communication applications like UART. This work validates the selection of AES-128 as the encryption standard for this project [4]. Hasija et al. (2023) presented a comprehensive survey of FPGA-based AES implementations, analyzing various architectural approaches to optimize performance. Their findings offer valuable insights into achieving high-speed and resource-efficient AES designs on FPGA platforms, which are directly applicable to this project [5]. Priyanka et al. (2023) provided detailed documentation of implementing AES on FPGA, emphasizing modularity and real-time encryption capabilities. Their research serves as a practical guide for the seamless integration of AES into secure UART systems [6]. Lu (2023) explored the theoretical and mathematical underpinnings of AES, emphasizing its cryptographic strength and resilience against common security breaches. This detailed analysis underlines the reliability of AES for securing sensitive data during transmission [7].

The design and hardware optimization of AES have been extensively studied, especially for FPGA applications. Swayamprakash et al. (2023) demonstrated the implementation of AES using Verilog HDL, focusing on modular design and hardware resource optimization. Their approach ensures scalability and compatibility with FPGA systems, providing critical insights into designing efficient AES modules [8]. Kinshuk et al. (2024) addressed the specific integration of AES with UART systems, demonstrating how this combination enhances data confidentiality and system-level security. Their research aligns directly with the objectives of this project, offering a real-world example of AES-128 implementation in UART communication for high-security applications [9].

The integration of encryption algorithms with UART has been successfully applied to various use cases. Sharmila et al. (2023) conducted a case study on the hardware implementation of AES for encrypted image transfer over UART, showcasing the feasibility of integrating secure encryption into serial communication systems. Their findings demonstrate that hardware-based encryption can effectively secure data transmission, even for high-bandwidth applications [10]. Cupi (2024) analyzed vulnerabilities in serial communication systems and proposed encryption as a solution to mitigate security risks. This research underscores the importance of integrating robust cryptographic protocols like AES into UART systems to address modern cybersecurity challenges [11].

Collectively, these studies provide a comprehensive framework for understanding UART communication, AES encryption, and their integration on FPGA platforms. They highlight the critical considerations for optimizing hardware resources, achieving high-speed encryption, and addressing potential vulnerabilities in secure communication systems. This body of knowledge forms a strong foundation for implementing the AES-128 algorithm for secure UART communication on the Artix-7 FPGA, ensuring both hardware efficiency and data confidentiality.

1.2 Motivation for the Project

The increasing reliance on embedded systems and IoT devices in critical applications such as healthcare, automotive, and industrial automation has amplified the need for secure and reliable communication protocols. UART, a widely used communication standard, is integral to such systems due to its simplicity, low resource requirements, and ease of implementation. However, the inherent lack of encryption in UART makes it vulnerable to data interception and tampering, posing a significant risk to sensitive information.

The Advanced Encryption Standard (AES), particularly the AES-128 variant, is recognized globally for its robust security features and computational efficiency. By integrating AES encryption with UART communication, this project aims to address the growing demand for secure data transmission in embedded and IoT applications. Implementing this system on the Artix-7 FPGA offers additional advantages such as real-time processing, high performance, and scalability, ensuring that the solution is both effective and adaptable.

The choice of the Artix-7 FPGA is motivated by its widespread adoption in academic and industrial applications, offering a balance between cost, performance, and hardware capabilities. Furthermore, the combination of UART and AES-128 encryption aligns with modern cybersecurity requirements, paving the way for a versatile and secure communication framework.

This project not only provides a practical solution to secure UART communication but also contributes to advancing knowledge in hardware-based cryptographic implementations. By addressing critical security gaps in UART communication and demonstrating the feasibility of real-time encryption on FPGA platforms, this work seeks to enhance the reliability and confidentiality of data transmission in modern embedded systems.

1.3 Work Flow

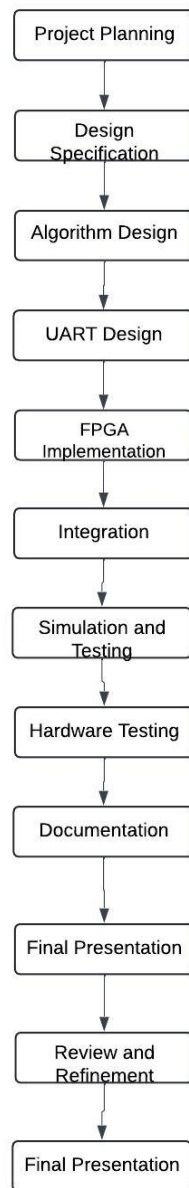


Figure 1.1: Work Flow

The following steps outline the process for the project, likely involving FPGA development and UART design.

1. Project Planning:

- Involves identifying the project goals, timeline, resources, and constraints.
- Define the problem statement and determine the scope of the work.
- Allocate team roles and responsibilities.

2. Design Specification:

- Write detailed specifications for the project, including hardware and software requirements.
- Define the inputs, outputs, and performance metrics for the system.

3. Algorithm Design:

- Develop the algorithms necessary for the project, such as encryption or data processing.
- Design high-level system workflows and processes.
- Use tools like flowcharts or pseudocode for clarity.

4. UART Design:

- Focus on the design of the UART communication system.
- Define the baud rate, data format (e.g., parity bits, stop bits), and protocols.
- Create a functional UART module capable of transmitting and receiving data.

5. FPGA Implementation:

- Write the RTL (Register Transfer Level) code for the design, integrating the UART and other components.
- Use HDL languages like Verilog or VHDL to program the FPGA.
- Simulate and synthesize the design for the FPGA.

6. Integration:

- Combine all the individual modules (e.g., UART, algorithm, control logic) into a unified system.
- Ensure all interfaces and communication paths work seamlessly.

7. Simulation and Testing:

- Perform software-based simulations to validate the design logic.
- Debug and test the system in a virtual environment to identify and fix errors.

8. Hardware Testing:

- Test the design on actual hardware, such as an FPGA board.
- Validate real-world performance, including data transmission, clock synchronization, and signal integrity.

9. Documentation:

- Record all aspects of the project, including system design, challenges, solutions, and results.

- Create user manuals and technical documentation.

10. Final Presentation:

- Prepare a detailed presentation of the project outcomes, methodologies, and conclusions.
- Include demonstrations of the working system, if applicable.

11. Review and Refinement:

- Gather feedback from peers or supervisors.
- Refine the design and implementation based on suggestions to improve performance or functionality.

12. Final Presentation (Revised):

- Present the refined project with improvements, incorporating feedback.
- Highlight the updates and discuss the final system performance.

Chapter 2: UART

UART (Universal Asynchronous Receiver/Transmitter) communication is a widely used protocol for serial data transmission, enabling data exchange between two devices with minimal hardware requirements. The fundamental principle of UART communication involves converting parallel data from a controlling device, such as a CPU, into a serial format for transmission. This serialized data is sent from the transmitting UART's Tx (transmit) pin to the receiving UART's Rx (receive) pin over a single wire. Once received, the serial data is converted back into parallel form by the receiving UART, making it accessible to the receiving device. The entire communication process requires only two wires—one for data transmission (Tx to Rx) and one for grounding—making UART a simple and efficient solution for serial communication.

A key characteristic of UART communication is its asynchronous nature, which means it does not rely on a shared clock signal to synchronize data transmission and reception. Instead, synchronization is achieved through the use of framing bits added to each data packet. Specifically, the transmitting UART inserts a *start bit* at the beginning and one or more *stop bits* at the end of the data packet. These bits act as markers, signaling the receiving UART when to start and stop interpreting the incoming data. The absence of a shared clock simplifies the hardware design but requires precise timing for successful data transfer.

The timing of data sampling by the receiving UART is governed by the **baud rate**, which is a measure of data transmission speed expressed in bits per second (bps). Both UARTs must operate at approximately the same baud rate for effective communication. A mismatch of more than 10% in baud rate between the transmitting and receiving devices can lead to misalignment of data bits, resulting in errors. This emphasizes the need for careful configuration of both devices to ensure synchronized timing.

In addition to matching baud rates, the two UART devices must also share the same *data packet structure*. This structure defines critical parameters such as the number of data bits, parity bits (used for error checking), and the number of stop bits. These settings must be configured identically on both UARTs to guarantee that the data is transmitted and interpreted correctly. Any inconsistency in these parameters can result in communication failure.

UART's simplicity and robustness make it a popular choice for many embedded systems and electronic applications. Its ability to transmit data reliably with minimal hardware requirements, combined with its flexibility to support various data formats and speeds, makes it an essential protocol in modern electronics. Despite its asynchronous nature, the use of start and stop bits ensures that data packets are framed and delivered accurately, providing an efficient mechanism for serial communication across a wide range of devices.

2.1 How UART Works

In UART communication, the transmitting UART receives data from a data bus, which is typically connected to a device like a CPU, memory, or microcontroller. This data is transferred to the transmitting UART in parallel form. Once the UART receives the parallel data, it processes it to create a data packet. This involves appending additional framing bits, including a *start bit*, a *parity bit* (if used), and one or more *stop bits*. These framing bits ensure that the data can be transmitted and interpreted correctly by the receiving UART.

After forming the data packet, the transmitting UART sends it serially, bit by bit, through its Tx (transmit) pin. On the other end, the receiving UART reads the incoming data packet serially, bit by bit, through its Rx (receive) pin. The receiving UART then reconstructs the original data by converting it back into parallel form and removing the start, parity, and stop bits. Finally, the reconstructed data is sent in parallel to the data bus on the receiving end, making it accessible to the intended device. This process ensures efficient and reliable data transfer between two devices using UART communication.

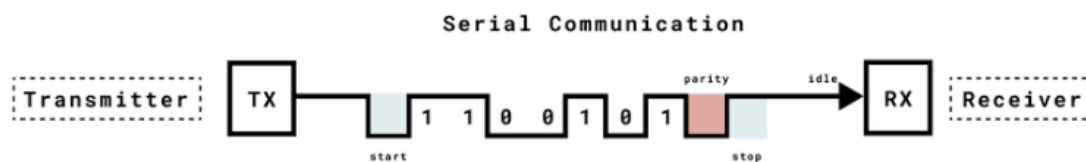


Figure 2.1: UART Serial Communication

2.2 Frame Format

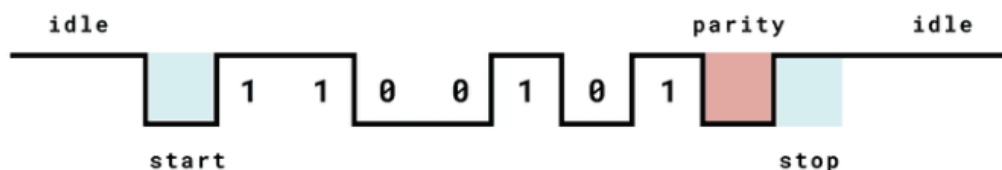


Figure 2.2: UART Frame Format

2.2.1 Start Bit

In UART communication, a **single start bit** is transmitted at the beginning of each data frame. The primary purpose of the start bit is to signal the start of data transmission and to alert the receiver that a new data frame is about to be sent.

The start bit is always **logic low (0)**, which means it is transmitted as a voltage level lower than the logic high threshold, typically around ground (0V). This low voltage level is recognized by the receiver as the start of the data packet.

Once the receiver detects the start bit, it knows that a new data frame is beginning. The receiver is then synchronized to begin receiving the subsequent bits of the data frame, ensuring accurate reception of the entire packet.

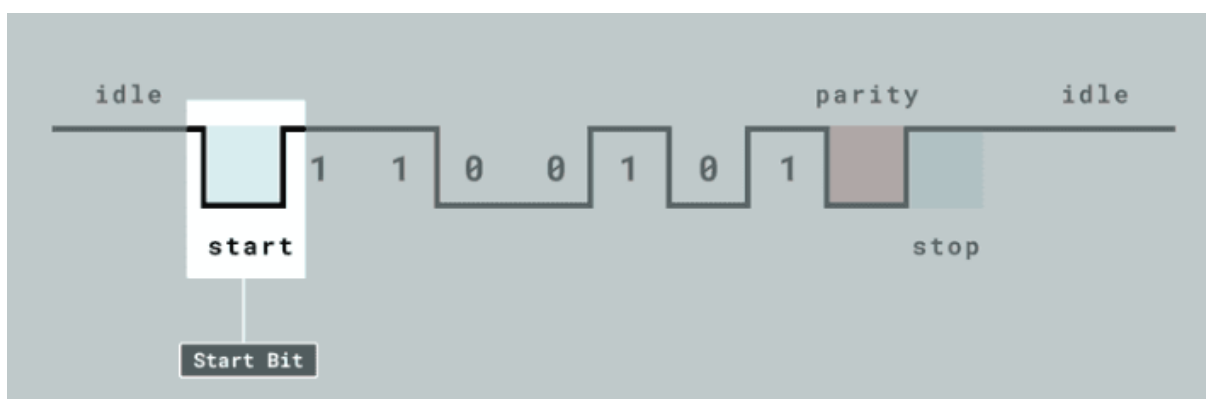


Figure 2.3: UART Start bit

2.2.2 Data bit

Data bits are a fundamental component of UART communication, as they carry the actual information to be transmitted. The number of data bits in a UART frame can vary, but a common and widely used configuration is **8 bits**. However, UART supports various character sizes, including **7-bit** and **6-bit** configurations, depending on the specific application requirements.

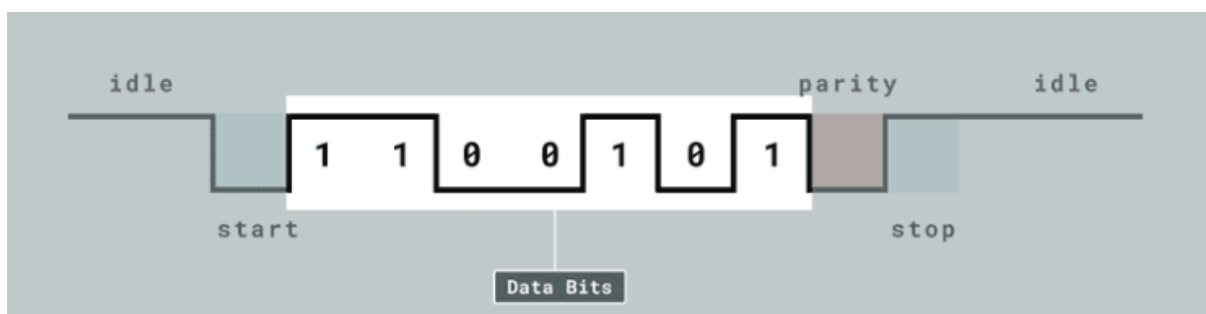


Figure 2.4: UART Data bit

2.2.3 Parity bit

In addition to data bits, UART communication may include a **parity bit** as part of the data frame. Parity is an error-checking mechanism that helps detect data transmission errors. Parity can be set to **odd** or **even**, and it ensures that the total number of bits set to logic "1" in a character is either even or odd, depending on the chosen parity type.

The presence of a parity bit allows the receiver to verify the integrity of the received data. If the number of "1" bits does not match the expected parity, an error is detected.

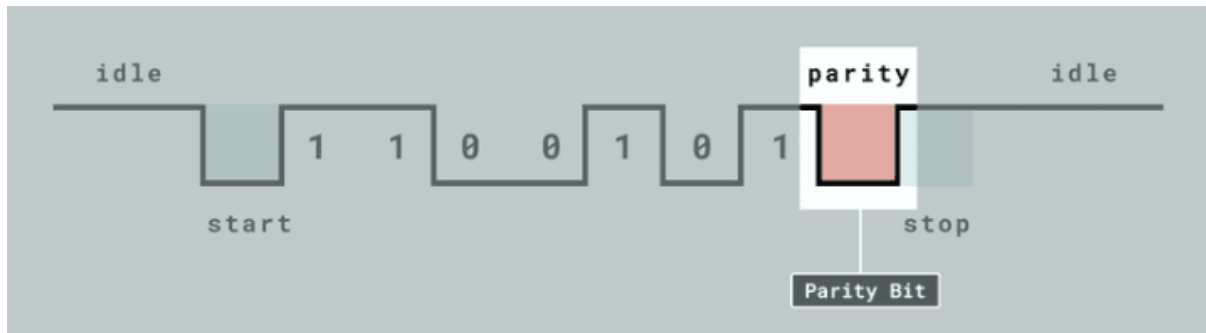


Figure 2.5: UART Parity bit

2.2.4 Stop bit

One or more **stop bits** are sent after the data bits within each UART frame. The stop bit(s) signal the end of the data byte and serve to indicate the conclusion of data transmission. The most common configuration is to use **one stop bit**, but in situations where added reliability is required, **two stop bits** can be employed.

The polarity of the stop bit(s) can vary, with some systems using a **high stop bit** and others using a **low stop bit**, based on the specific UART configuration.

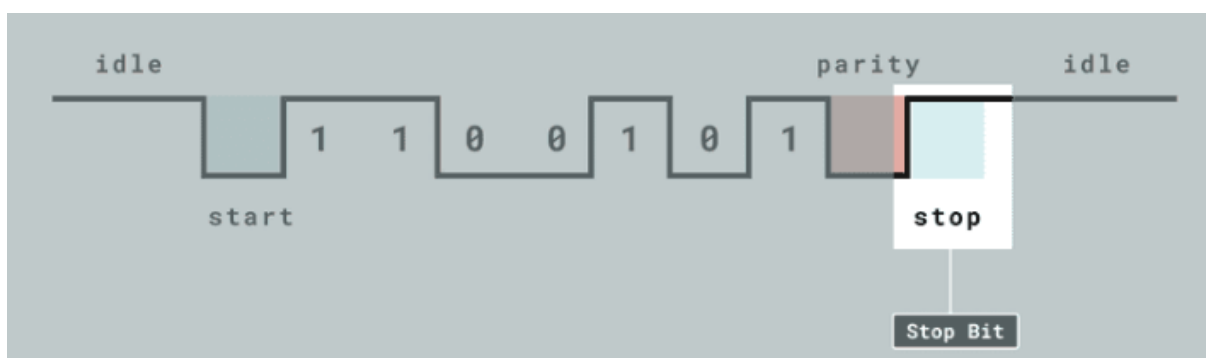


Figure 2.6: UART Stop bit

2.3 Challenges in securing Uart

Securing UART communication is essential, especially in environments where data security is a priority, such as in embedded systems or IoT devices. Below are key security challenges from the perspective of protecting against theft and unauthorized access:

2.3.1 Data Interception

Since UART transmits data in plain text without encryption, attackers can easily intercept the data using a physical connection or by tapping into the transmission line. This exposes sensitive information to theft. To prevent this, encryption algorithms like AES can be implemented, but this introduces additional complexity and resource demands on the system.

2.3.2 Lack of Authentication

UART doesn't support authentication, making it vulnerable to impersonation attacks. An attacker could masquerade as a legitimate device and intercept or inject malicious data. Adding authentication mechanisms, such as shared keys or digital signatures, can prevent unauthorized devices from connecting, but this requires additional software or hardware support.

2.3.3 Physical Access

UART systems are often vulnerable to physical access, where an attacker can directly connect to the Tx and Rx lines to steal data. This is particularly problematic in unsecured environments. Physical security measures, such as using secure connectors, isolating the communication lines, or employing encryption at the hardware level, are necessary to mitigate this risk.

2.3.4 Data Integrity

Without robust error-checking mechanisms beyond basic parity bits, UART is susceptible to data corruption, which can be exploited by attackers to alter data. Implementing stronger data integrity measures, such as checksums or cryptographic hashes, ensures that data remains intact and is not tampered with during transmission.

2.3.5 Insecure Default Settings

Many UART devices come with insecure default configurations that can be exploited by attackers. For example, weak baud rates or lack of encryption may allow unauthorized access. Ensuring that all UART settings are configured securely and using strong encryption defaults is critical to protect against potential theft or unauthorized data access.

Chapter 3: AES Algorithm

3.1 Introduction to AES

The Advanced Encryption Standard (AES) is a robust and widely adopted symmetric key encryption algorithm designed to securely encrypt and decrypt data. Established by the National Institute of Standards and Technology (NIST) in 2001, AES was introduced as a replacement for the older Data Encryption Standard (DES) to address emerging security challenges. Its high efficiency and strong cryptographic foundation make it suitable for diverse applications, including government communication, banking, and secure online transactions.

AES encrypts data in fixed-size blocks of 128 bits (16 bytes) and relies on symmetric encryption, where the same key is used for both encryption and decryption processes. This feature ensures simplicity and speed while maintaining high levels of security.

3.1.1 AES Key Sizes and Encryption Rounds

AES supports three key sizes, each providing varying levels of security and computational complexity:

- **AES-128:** Uses a 128-bit key and performs 10 rounds of encryption.
- **AES-192:** Uses a 192-bit key and performs 12 rounds of encryption.
- **AES-256:** Uses a 256-bit key and performs 14 rounds of encryption.

Each encryption round involves a series of transformation steps designed to obscure the original data and enhance security. The number of rounds directly correlates with the key length, with longer keys requiring additional rounds to ensure greater resilience against attacks.

- **AES-128:** 10 rounds
- **AES-192:** 12 rounds
- **AES-256:** 14 rounds

In this project, we have used a **128-bit key**, so the encryption process includes **10 rounds**, ensuring a balance between computational efficiency and security. This configuration is ideal for embedded systems and applications where resource optimization is critical.

3.1.2 AES Block Structure

AES operates on fixed-size blocks of data, specifically 128 bits (16 bytes). This block is organized into a **4×4 matrix of bytes**, commonly referred to as the **state**. Each element of this matrix represents a byte (8 bits), which is typically displayed in a two-digit hexadecimal format. This structure forms the foundation of the encryption and decryption processes in AES, enabling efficient computation and transformation of data.

For example, a 128-bit block of plaintext might be represented as:

Plaintext (in hexadecimal): {A0, A1, A2, A3, A4, A5, A6, A7, A8, A9, AA, AB, AC, AD, AE, AF}

This is then arranged into a 4×4 state matrix as follows:

A0	A4	A8	AC
A1	A5	A9	AD
A2	A6	AA	AE
A3	A7	AB	AF

Figure 3.1: AES state Matrix

3.1.3 Significance of the State Matrix

- **Row and Column Operations:** AES applies various transformations during encryption and decryption, such as **SubBytes**, **ShiftRows**, **MixColumns**, and **AddRoundKey**, directly to this state matrix.
- **Hexadecimal Representation:** Each byte is represented in hexadecimal to simplify operations and align with cryptographic standards.
- **Efficient Computation:** The matrix structure facilitates parallel processing of rows and columns, making AES both computationally efficient and secure.

This 4×4 state matrix is transformed through several encryption rounds, depending on the key size, ultimately producing the ciphertext in a similarly arranged format.

3.1.4 Key Expansion Process

Before encryption, the AES key is expanded into a set of round keys. This process generates a key schedule from the original key, which is used in the **AddRoundKey** step in each round of encryption.

The key expansion involves taking the 128-bit key and expanding it into a larger set of key words. Each round requires a round key, and the round key for each round is derived from the previous round's key.

For a 128-bit key (AES-128), the key expansion creates 44 words (each word is 4 bytes). For AES-192, 52 words are generated, and for AES-256, 60 words are generated. The number of words depends on the number of rounds as shown below:

- **AES-128:** 44 words (for 10 rounds)
- **AES-192:** 52 words (for 12 rounds)
- **AES-256:** 60 words (for 14 rounds)

K0	K4	K8	KC
K1	K5	K9	KD
K2	K6	KA	KE
K3	K7	KB	KF

Figure 3.2: AES Key

3.2 AES Encryption Process

AES (Advanced Encryption Standard) is a symmetric encryption algorithm that operates on blocks of data (128 bits) and uses a key size of 128 bits. The encryption process involves several rounds of operations, depending on the key size. For AES-128, it uses 10 rounds of operations.

3.2.1 Add RoundKey

AddRoundKey is a simple XOR operation where the current state is XORed with a round key. Each round key is derived from the original encryption key during the key expansion phase, and the round keys are used in each round of the AES algorithm. In this step, the round key is applied to the state matrix using bitwise XOR.

Operation: Each byte of the state is XORed with the corresponding byte in the round key.

Purpose: This operation ensures that the encryption process uses the secret key at each step, making it dependent on the key and ensuring the encryption is key-dependent.

A0	A4	A8	AC
A1	A5	A9	AD
A2	A6	AA	AE
A3	A7	AB	AF

 $\oplus$

K0	K4	K8	KC
K1	K5	K9	KD
K2	K6	KA	KE
K3	K7	KB	KF

 $=$

S0	S4	S8	SC
S1	S5	S9	SD
S2	S6	SA	SE
S3	S7	SB	SF

Figure 3.3: ADDRound Key

3.2.2 SubBytes

SubBytes is a substitution step that operates on each byte in the state. During this step, each byte in the 4x4 matrix (state) is substituted with a corresponding byte from the S-box, a non-linear substitution table. The S-box is designed to provide confusion, making the relationship between the plaintext and ciphertext as complex as possible.

Operation: For each byte of the state matrix, its value is replaced by the corresponding value in the S-box.

		y															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
	1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
	2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
	3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
	4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
	5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
	6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
	7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
	8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
	9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
	a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
	b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
	c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
	d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
	e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
	f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

Figure 3.4: S box

Purpose: The primary goal of this operation is to introduce non-linearity and confusion into the encryption process.

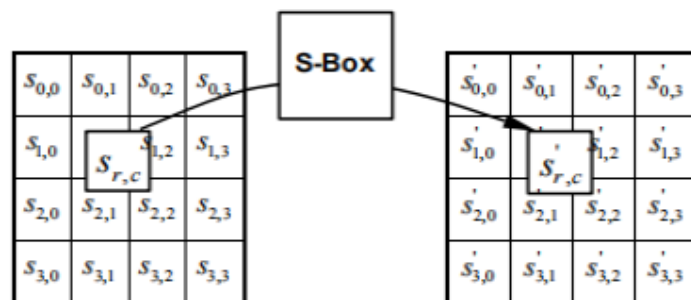


Figure 3.5: SubBytes

3.2.3 ShiftRows

ShiftRows is a transposition step that shifts the rows of the state matrix to the left by a certain number of positions. The rows are shifted as follows:

- **Row 0:** No shift (it stays the same).
- **Row 1:** Shifted by 1 byte to the left.
- **Row 2:** Shifted by 2 bytes to the left.
- **Row 3:** Shifted by 3 bytes to the left.

This operation mixes the bytes in each column of the state matrix, ensuring that bytes from different columns interact. It provides diffusion by spreading the influence of each byte across the entire state matrix.

Purpose: This operation enhances diffusion by making sure that bytes from different columns are interspersed, making it harder to detect patterns in the data.

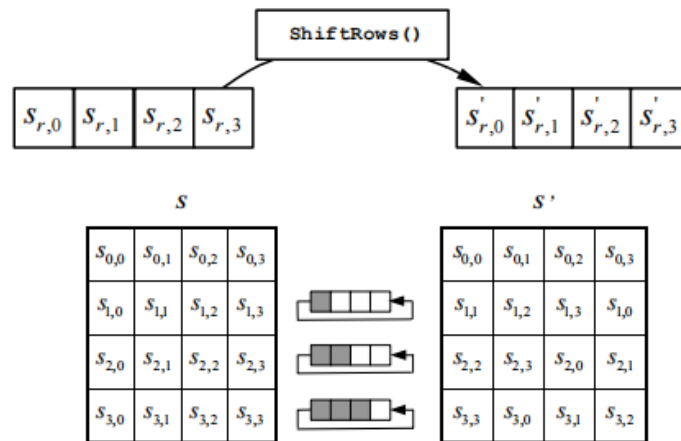


Figure 3.6: ShiftRows

3.2.4 MixColumns

MixColumns is a column-wise operation that mixes the data in each column of the state. It operates on each column as a four-byte vector and performs a linear transformation, using a fixed matrix to multiply the column. The transformation involves Galois Field (GF) multiplication. The operation combines the bytes in each column, providing more diffusion.

Operation: Each column is treated as a four-element vector, and matrix multiplication is performed using a fixed matrix (often referred to as the MixColumns matrix). The result is a new column with mixed values.

Purpose: The goal of MixColumns is to further spread the influence of each byte across the entire column, making it harder to discern any patterns in the ciphertext.

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}$$

Figure 3.7: MixColumns

3.2.5 Flow Chart of Encryption

1. Initial Rounds:

- **AddRoundKey:** The state is XORed with the initial round key.

2. Main Rounds (9 Rounds):

- **SubBytes:** Each byte in the state is substituted using the S-box.
- **ShiftRows:** The rows of the state matrix are shifted.
- **MixColumns:** Each column of the state matrix is mixed using a fixed matrix.
- **AddRoundKey:** The round key is XORed with the state.

3. Final Round (10th Round):

- **SubBytes:** Each byte in the state is substituted using the S-box.
- **ShiftRows:** The rows of the state matrix are shifted.
- **AddRoundKey:** The round key is XORed with the state (final round).

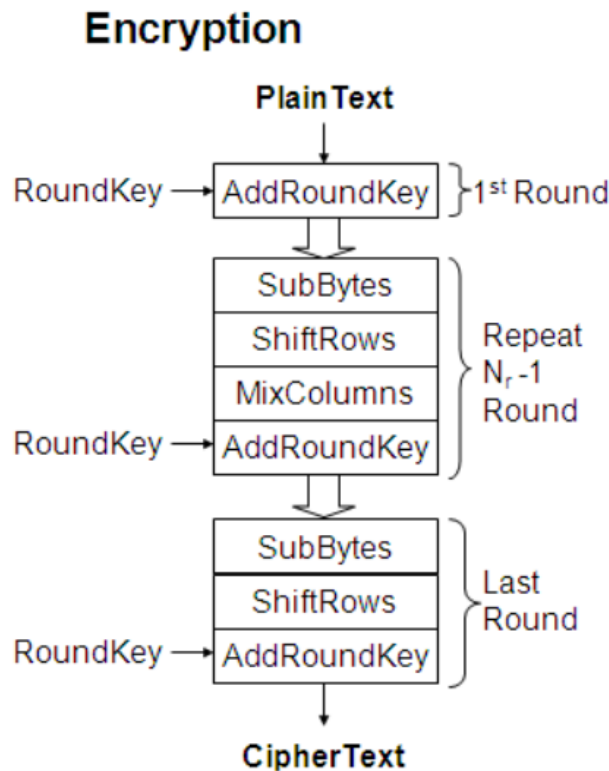


Figure 3.8: Encryption Flowchart

3.3 AES Decryption Process

The AES (Advanced Encryption Standard) decryption process is the reverse operation of the encryption process. It takes the encrypted ciphertext and reconstructs the original plaintext using the same secret key that was used during encryption. This symmetry in key usage is a characteristic of symmetric-key cryptographic algorithms like AES.

In the decryption process, the sequence of transformations applied during encryption is performed in reverse order. However, these transformations are not simply undone but are mathematically inverted to ensure data integrity and security. The main steps in the AES decryption process include:

3.3.1 ADD Round Key

AddRoundKey is the same in both encryption and decryption. During decryption, it involves XORing the state matrix with the round key in reverse order. This step ensures that the key used in encryption is effectively undone.

The round key is XORed with the state matrix at each round. This step is performed in reverse order, starting from the last round key.

Purpose:

- To reverse the round key mixing performed during encryption.

- Restores the state matrix to its pre-encrypted form by applying the round keys in reverse order.

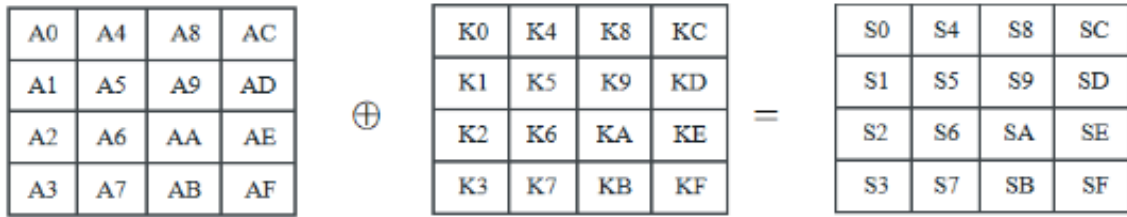


Figure 3.9: ADD RoundKey

3.3.2 Inverse ShiftRows

Inverse ShiftRows reverses the row shifting applied in the ShiftRows operation during AES encryption. The rows of the state matrix are shifted to the right, restoring the original arrangement of the bytes.

Row 0: No shift (it stays the same).

Row 1: Shifted by 1 byte to the right.

Row 2: Shifted by 2 bytes to the right.

Row 3: Shifted by 3 bytes to the right.

Purpose:

- To reverse the left shifts applied in ShiftRows during encryption.
- Ensures the state matrix is restored to its original form.

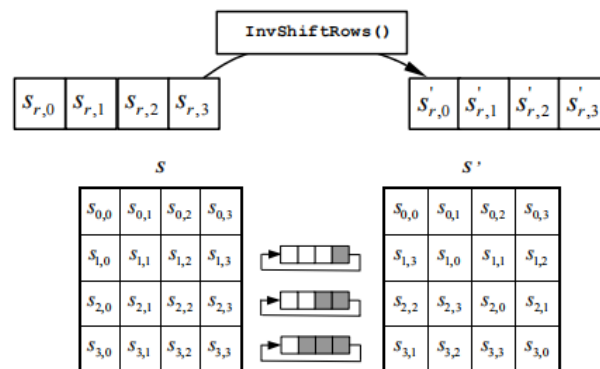


Figure 3.10: Inverse ShiftRows

3.3.3 Inverse SubBytes

Inverse SubBytes is the reverse operation of the SubBytes step in AES encryption. In this step, each byte in the state matrix is substituted with its corresponding value from the inverse S-box, reversing the byte substitution done during encryption.

Each byte in the state is replaced by its inverse from the inverse S-box.

		y															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
	1	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
	2	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
	3	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
	4	72	f8	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
	5	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
	6	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
	7	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
	8	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	f0	b4	e6	73
	9	96	ac	74	22	e7	ad	35	85	e2	f9	37	e8	1c	75	df	6e
	a	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
	b	fc	56	3e	4b	c6	d2	79	20	9a	db	c0	fe	78	cd	5a	f4
	c	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
	d	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
	e	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
	f	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

Figure 3.11: Inverse S-box

This step reverses the non-linear byte substitution used during encryption.

Purpose:

- To reverse the byte substitution performed in AES encryption.
- Restores the state matrix to its original form before the SubBytes operation.

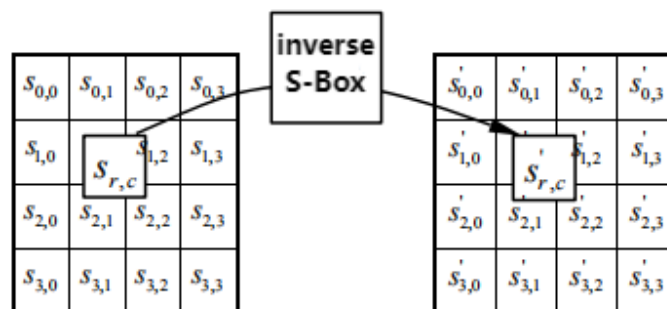


Figure 3.12: Inverse SybBytes

3.3.4 Inverse MixColumns

Inverse MixColumns undoes the column mixing applied during the MixColumns step in AES encryption. This step uses the inverse MixColumns matrix to reverse the diffusion and restore the original byte values in each column of the state matrix.

Each column in the state matrix is multiplied by the inverse MixColumns matrix.

This step reverses the column mixing used to spread the influence of each byte across the matrix.

Purpose:

- To reverse the column mixing performed during encryption.
- Restores the state matrix's column values to their original form.

$$\begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix} = \begin{bmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}$$

Figure 3.13: Inverse MixColumns

3.3.5 Flow Chart of Decryption

1. Initial Rounds

- **AddRoundKey:** XOR the ciphertext with the round key from the last round (reverse order).

2. Main Round(9 Rounds)

- **Inverse MixColumns:** Apply the inverse MixColumns transformation to the state matrix.
- **Inverse ShiftRows:** Shift rows to the right by the appropriate number of bytes.
- **Inverse SubBytes:** Substitute bytes using the inverse S-box.
- **AddRoundKey:** XOR the state matrix with the round key for the current round.

3. Final Round (10th Round)

- **Inverse ShiftRows:** Shift rows to the right by the appropriate number of bytes.
- **Inverse SubBytes:** Substitute bytes using the inverse S-box.
- **AddRoundKey:** XOR the state matrix with the final round key.

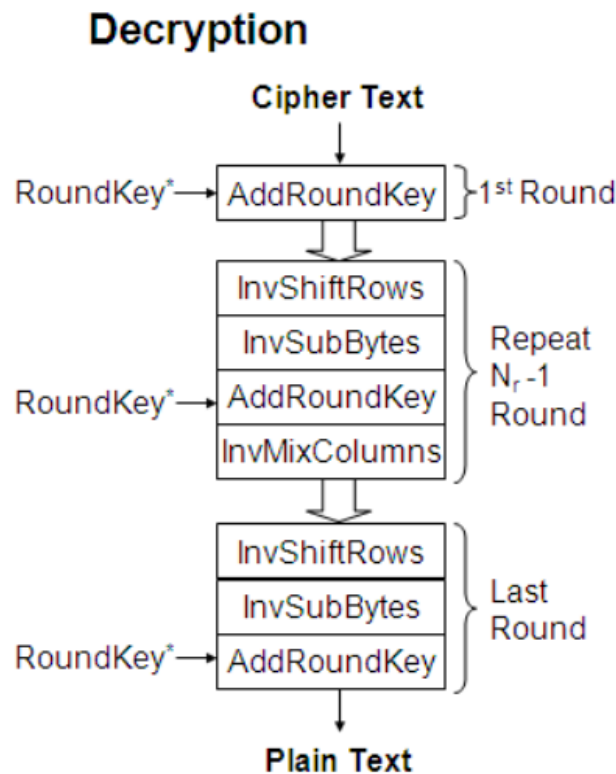


Figure 3.14: Decryption Flow Chart

3.4 AES Encryption Example

In this AES encryption example,

input plaintext 32 43 f6 a8 88 5a 30 8d 31 31 98 a2 e0 37 07 34 is encrypted using a

128-bit cipher key 2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c. The process involves key expansion, substitution, shifting, mixing, and multiple rounds of encryption to produce a secure ciphertext.

Round Number	Start of Round	After SubBytes	After ShiftRows	After MixColumns	Round Key Value
input	32 88 31 e0 43 5a 31 37 f6 30 98 07 a8 8d a2 34				2b 28 ab 09 7e ae f7 cf 15 d2 15 4f 16 a6 88 3c
					⊕ =
	19 a0 9a e9 3d f4 c6 f8 e3 e2 8d 48 b2 2b 2a 08	d4 e0 b8 1e 27 bf b4 41 11 98 5d 52 ae f1 e5 30	d4 e0 b8 1e bf b4 41 27 5d 52 11 98 30 ae f1 e5	04 e0 48 28 66 cb f8 06 81 19 d3 26 e5 9a 7a 4c	a0 88 23 2a fa 54 a3 6c fe 2c 39 76 17 b1 39 05
					⊕ =
	a4 68 6b 02 9c 9f 5b 6a 7f 35 ea 50 f2 2b 43 49	49 45 7f 77 de db 39 02 d2 96 87 53 89 f1 1a 3b	49 45 7f 77 db 39 02 de 87 53 d2 96 3b 89 f1 1a	58 1b db 1b 4d 4b e7 6b ca 5a ca b0 f1 ac a8 e5	f2 7a 59 73 c2 96 35 59 95 b9 80 f6 f2 43 7a 7f
					⊕ =
1	aa 61 82 68 8f dd d2 32 5f e3 4a 46 03 ef d2 9a	ac ef 13 45 73 c1 b5 23 cf 11 d6 5a 7b df b5 b8	ac ef 13 45 c1 b5 23 73 d6 5a cf 11 b8 7b df b5	75 20 53 bb ec 0b c0 25 09 63 cf d0 93 33 7c dc	3d 47 1e 6d 80 16 23 7a 47 fe 7e 88 7d 3e 44 3b
					⊕ =
	48 67 4d d6 6c 1d a3 5f 4e 9d b1 58 ee 0d 38 e7	52 85 e3 f6 50 a4 11 cf 2f 5e c8 6a 28 d7 07 94	52 85 e3 f6 a4 11 cf 50 c8 6a 2f 5e 94 28 d7 07	0f 60 6f 5e d6 31 c0 b3 da 38 10 13 a9 bf 6b 01	ef a8 b6 db 44 52 71 0b a5 5b 25 ad 41 7f 3b 00
					⊕ =
	e0 c8 d9 85 92 63 b1 b8 7f 63 35 be e8 c0 50 01	e1 e8 35 97 4f fb c8 6c d2 fb 96 ae 9b ba 53 7c	e1 e8 35 97 fb c8 6c 4f 96 ae d2 fb 7c 9b ba 53	25 bd b6 4c d1 11 3a 4c a9 d1 33 c0 ad 68 8e b0	d4 7c ca 11 d1 83 f2 f9 c6 9d b8 15 f8 87 bc bc
					⊕ =
2	f1 c1 7c 5d 00 92 c8 b5 6f 4c 8b d5 55 ef 32 0c	a1 78 10 4c 63 4f e8 d5 a8 29 3d 03 fc df 23 fe	a1 78 10 4c 4f e8 d5 63 3d 03 a8 29 fe fc df 23	4b 2c 33 37 86 4a 9d d2 8d 89 f4 18 6d 80 e8 d8	6d 11 db ca 88 0b f9 00 a3 3e 86 93 7a fd 41 fd
					⊕ =
	26 3d e8 fd 0e 41 64 d2 2e b7 72 8b 17 7d a9 25	f7 27 9b 54 ab 83 43 b5 31 a9 40 3d f0 ff d3 3f	f7 27 9b 54 83 43 b5 ab 40 3d 31 a9 3f f0 ff d3	14 46 27 34 15 16 46 2a b5 15 56 d8 bf ec d7 43	4e 5f 84 4e 54 5f a6 a6 f7 c9 4f dc 0e f3 b2 4f
					⊕ =
	5a 19 a3 7a 41 49 e0 8c 42 dc 19 04 b1 1f 65 0c	be d4 0a da 83 3b e1 64 2c 86 d4 f2 c8 c0 4d fe	be d4 0a da 3b e1 64 83 d4 f2 2c 86 fe c8 c0 4d	00 b1 54 fa 51 c8 76 1b 2f 89 6d 99 d1 ff cd ea	ea b5 31 7f d2 8d 2b 8d 73 ba f5 29 21 d2 60 2f
					⊕ =
3	ea 04 65 85 83 45 5d 96 5c 33 98 b0 f0 2d ad c5	87 f2 4d 97 ec 6e 4c 90 4a c3 46 e7 8c d8 95 a6	87 f2 4d 97 6e 4c 90 ec 46 e7 4a c3 a6 8c d8 95	47 40 a3 4c 37 d4 70 9f 94 e4 3a 42 ed a5 a6 bc	ac 19 28 57 77 fa d1 5c 66 dc 29 00 f3 21 41 6e
					⊕ =
	eb 59 8b 1b 40 2e a1 c3 f2 38 13 42 1e 84 e7 d2	e9 cb 3d af 09 31 32 2e 89 07 7d 2c 72 5f 94 b5	e9 cb 3d af 31 32 2e 09 7d 2c 89 07 b5 72 5f 94		d0 c9 e1 b6 14 ee 3f 63 f9 25 0c 0c a8 89 c8 a6
					⊕ =
	39 02 dc 19 25 dc 11 6a 84 09 85 0b 1d fb 97 32				
output					

Figure 3.15: AES Example

Chapter 4: Integration OF AES in UART

This section describes the integration of the AES-128 encryption algorithm for secure UART communication implemented on the Artix-7 FPGA. The system ensures secure data transmission by incorporating both encryption and decryption processes using AES-128.

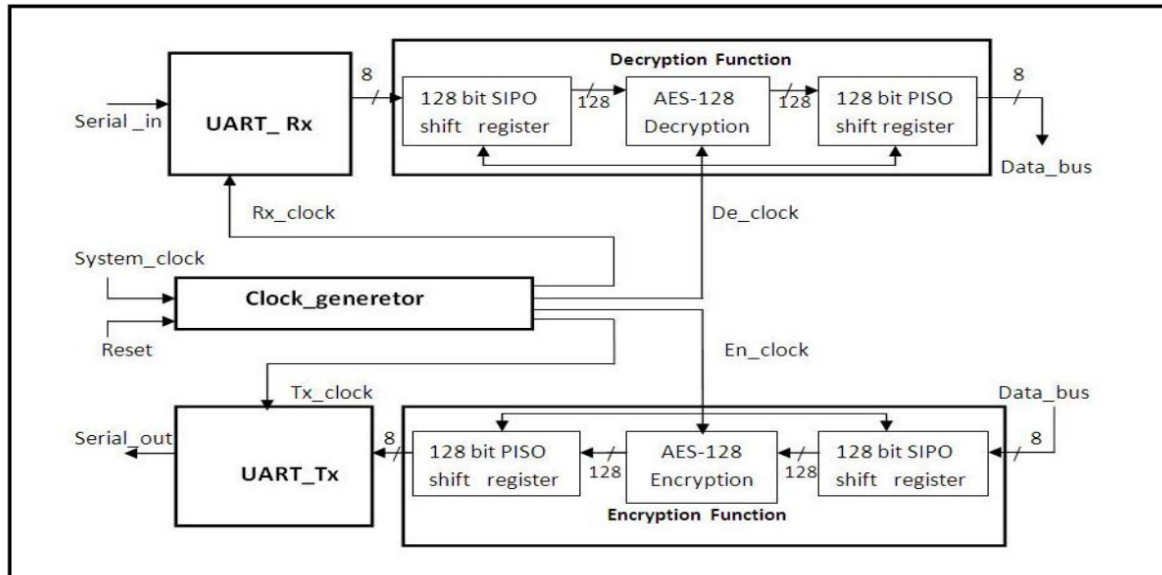


Figure 4.1: Integration OF AES in UART

4.1 Block Diagram Explanation

- **UART_Rx (Universal Asynchronous Receiver-Transmitter Receiver):** Receives serial input data (**Serial_in**) and outputs an 8-bit parallel data stream to the next block.
- **128-bit SIPO (Serial-In Parallel-Out) Shift Register:** Converts the 8-bit parallel data stream into a 128-bit parallel data stream.
- **AES-128 Decryption:** Decrypts the 128-bit data using the AES-128 decryption algorithm.
- **128-bit PISO (Parallel-In Serial-Out) Shift Register:** Converts the 128-bit parallel decrypted data back into an 8-bit parallel data stream.
- **Data_bus:** Carries the 8-bit parallel data stream to the next stage.
- **Clock_generator:** Generates various clock signals (**Rx_clock**, **Tx_clock**, **De_clock**, **En_clock**) from the system clock (**System_clock**) and reset signal (**Reset**).
- **UART_Tx (Universal Asynchronous Receiver-Transmitter Transmitter):** Transmits the serial output data (**Serial_out**) and receives an 8-bit parallel data stream from the previous block.

- **128-bit PISO Shift Register:** Converts the 8-bit parallel data stream into a 128-bit parallel data stream.
- **AES-128 Encryption:** Encrypts the 128-bit data using the AES-128 encryption algorithm.
- **128-bit SIPO Shift Register:** Converts the 128-bit parallel encrypted data back into an 8-bit parallel data stream.

4.2 Data Flow

1. Receiving Data:

- Data enters through UART_Rx, converting it from serial to 8-bit parallel format.
- The 8-bit data is accumulated into a 128-bit block by the SIPO shift register.

2. Decryption:

- The 128-bit block is decrypted using AES-128, converting it to plaintext.
- The plaintext 128-bit block is then converted back to an 8-bit parallel data stream by the PISO shift register.

3. Processing and Transmission:

- The 8-bit parallel data stream flows through the Data_bus for further processing or transmission.
- UART_Tx takes this data, converts it back to serial format, and sends it out as Serial_out.

4. Encryption for Secure Transmission:

- Before transmission, data is accumulated into 128-bit blocks using the PISO shift register.
- AES-128 encryption block encrypts these blocks for security.
- The encrypted data is converted back to an 8-bit parallel stream by the SIPO shift register.

5. Final Transmission:

- UART_Tx takes the 8-bit parallel encrypted data, converts it to serial format, and transmits it securely.

Chapter 5: Software Simulation

The software simulation for the AES encryption and decryption process was carried out using **Vivado 2014**, a robust and widely-used design suite by Xilinx for FPGA development. The primary objective of this simulation was to validate the correctness of the AES algorithm implementation at the functional level. A dedicated testbench was crafted to automate the testing process, ensuring consistency and repeatability across multiple test scenarios. This testbench served as a virtual testing environment, where a series of input test vectors, including 128-bit plaintext and 128-bit encryption keys, were applied to the AES module. The expected encrypted outputs and decrypted plaintexts were compared against actual results, allowing precise evaluation of the algorithm's behavior under various conditions.

Vivado's simulation tools provided an intuitive and comprehensive platform to observe signal waveforms, analyze timing behavior, and debug design issues in real-time. The suite's integrated environment allowed seamless compilation, elaboration, and simulation of the Verilog code, enabling quick iterations and refinements of the design. Its waveform viewer facilitated in-depth monitoring of signals, helping identify and rectify potential discrepancies between the expected and actual outputs. This iterative simulation process was critical for ensuring the reliability and robustness of the AES implementation before proceeding to hardware testing.

5.1 Testbench Description

The testbench was designed to generate five random 128-bit plaintext inputs and corresponding 128-bit random keys. These inputs were passed to the AES encryption module, and the encrypted outputs were subsequently decrypted by the AES decryption module. The decrypted data was compared to the original plaintext input to validate the correctness of the AES implementation.

5.2 Simulation Workflow

1. Input Generation

Random 128-bit plaintext and key pairs were generated within the testbench to simulate real-world scenarios and ensure robustness of the design.

2. Encryption

Each plaintext input was encrypted using the AES encryption module.

3. Decryption

The encrypted output was passed to the AES decryption module, producing the decrypted output.

4. Verification

The decrypted output was compared with the original plaintext input:

- If the decrypted output matched the plaintext, the simulation displayed a message: "Successful : Data received correctly."
- If a mismatch occurred, an error message was displayed: "Error: Data mismatched."

5.3 Results

The simulation consistently produced successful matches, verifying the correctness of the AES algorithm implementation. Any data mismatch was logged for further analysis, ensuring that all edge cases were accounted for during testing.

```
INFO: [Post-Place 57] Run Simulation Run AES 1000000
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:20 . Memory (MB): peak = 701.145 ; gain = 24.914
run_all
Data transmission and reception successful! Sent: b1f056638484d609c0895e8112153524, Received: b1f056638484d609c0895e8112153524, Key: 89375212b2c2846546df998d06b97b0d ,
Data transmission and reception successful! Sent: 1e8dcd3d3b23f17606d7cd0d00f3e301, Received: 1e8dcd3d3b23f17606d7cd0d00f3e301, Key: e33724c67cfde9f9462df78c76d457ed ,
Data transmission and reception successful! Sent: bbd2727772aff7e5d513d2aae2f784c5, Received: bbd2727772aff7e5d513d2aae2f784c5, Key: e77696ce793069f247ecdb8f8932d612 ,
Data transmission and reception successful! Sent: de8e28bd2e58495ce2ca4ec5f4007ae8, Received: de8e28bd2e58495ce2ca4ec5f4007ae8, Key: 0573870ablef6263b2a7266596ab582d ,
Data transmission and reception successful! Sent: cecccc9d557845aal0642120c03b2280, Received: cecccc9d557845aal0642120c03b2280, Key: a9a7d65386bc380d8983b813cb203e96 ,
$finish called at time : 11380 ns : File "F:/semester 7 subject/project/AES_UART/AES_UART.srcs/sim_1/new/tb_aes_uart.v" Line 101
```

Figure 5.1: Vivado Console Output

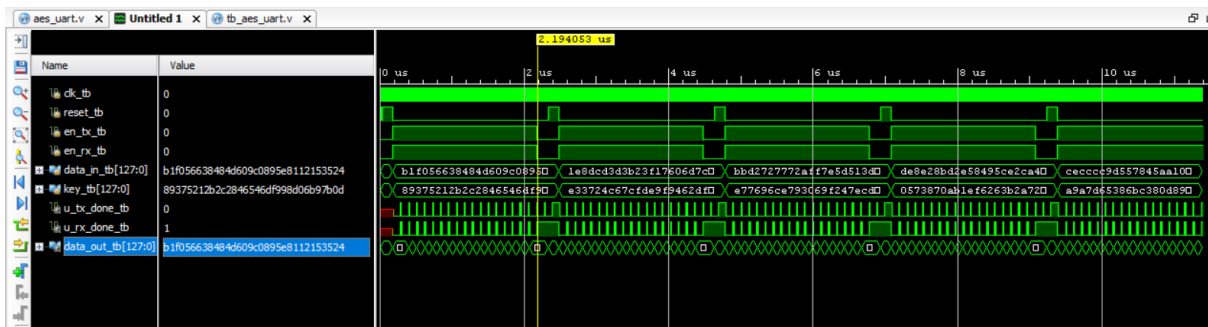


Figure 5.2: Vivado Simulation

5.4 RTL Schematic Description

The RTL schematic represents the hardware-level implementation of the AES encryption and decryption modules. It includes key components such as the S-box, shift rows, mix columns, and add round key operations, which are essential for AES. The schematic illustrates how data flows through these components, with inputs such as the plaintext and key entering the system and outputs being the encrypted or decrypted data. Control signals manage the sequencing of operations across multiple clock cycles, ensuring the proper execution of the AES rounds. The RTL design is optimized for performance, supporting secure and efficient data processing in hardware.

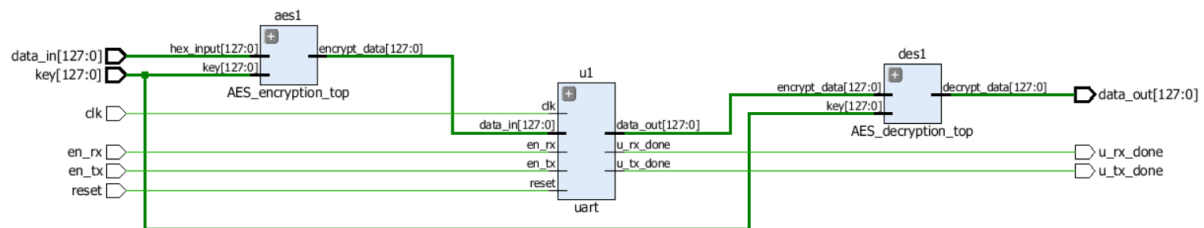


Figure 5.3: RTL Schematic

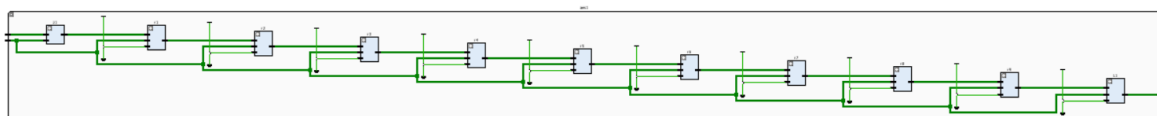


Figure 5.4: Inside AES encryption Module

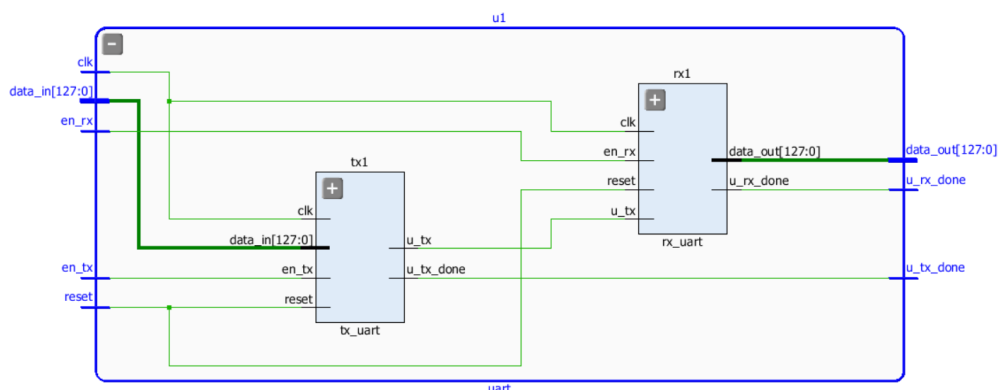


Figure 5.5: Inside Uart Module

Chapter 6: HardWare Implementation

The hardware implementation of the AES-128 algorithm for secure UART communication was carried out using two Artix-7 FPGA boards. To achieve seamless communication and encryption/decryption functionality, the overall design was divided into two distinct sections, with each board handling a specific part of the process.

Board 1: UART Receiver, AES Encryption, and UART Transmission

The first board performs the following operations:

- **UART Receiver (UART RX):** Data is received serially from the transmitting device (e.g., PC or another FPGA board) using the UART protocol.
- **AES Encryption:** The received 128-bit data is encrypted using the AES-128 encryption algorithm. The encryption module employs a 128-bit symmetric key, ensuring secure data transformation.
- **UART Transmitter (UART TX):** The encrypted data is transmitted serially to the second FPGA board via UART.

Board 2: UART Receiver, AES Decryption, and UART Transmission

The second board handles the following operations:

- **UART Receiver (UART RX):** Encrypted data from the first board is received serially via UART.
- **AES Decryption:** The received encrypted data is decrypted using the AES-128 decryption algorithm with the same 128-bit symmetric key used in encryption. The output is the original plaintext data.
- **UART Transmitter (UART TX):** The decrypted plaintext data is transmitted back to the originating device or another module for further processing.

6.1 Basys 3 FPGA Board

The Basys 3 FPGA board is an entry-level development platform designed by Digilent, equipped with a Xilinx Artix-7 FPGA (XC7A35T-1CPG236C). It provides an accessible and powerful platform for learning and prototyping digital circuits, making it ideal for both beginners and professionals in the field of digital design. The board operates at a clock frequency of 100 MHz and supports a variety of applications, including digital logic design, signal processing, and embedded systems.

Basys 3 features a rich set of onboard peripherals, including four 7-segment displays, 16 switches, 16 LEDs, and five push buttons, allowing for hands-on experimentation and debugging without additional hardware. It also includes USB ports for power and programming, an onboard oscillator, and a Pmod header for interfacing with external devices.

6.2 Data Flow and Encryption/Decryption Process

1. Data Input from PC to Board 1

- **PC Input (1 Byte at a Time):** The process initiates with the PC sending data to Board 1 over a UART connection. The PC transmits data one byte at a time, which is a common approach in serial communication due to its simplicity and efficiency.

2. Board 1 Processing (Encryption)

- **UART Receiver:** The UART receiver on Board 1 captures the incoming serial data from the PC. It processes the data byte by byte, ensuring that each byte is received sequentially without loss.
- **Serial-In Parallel-Out (SIPO):** As each byte is received, it is stored in a 128-bit SIPO shift register. This register accumulates 16 consecutive bytes from the UART, thereby forming a complete 128-bit word. This step is critical as the AES encryption algorithm requires a full 128-bit block for processing.
- **AES Encryption:** Once the SIPO register is filled with the 128 bits of data, the AES encryption module is activated. It encrypts the plaintext using a predefined 128-bit key, producing a 128-bit ciphertext. This encrypted data is essential for secure communication, ensuring that even if intercepted, the data remains unreadable without the key.
- **Parallel-In Serial-Out (PISO):** The next step involves transferring the encrypted 128-bit data to a PISO shift register. The PISO register converts the 128-bit parallel data into an 8-bit serial stream, preparing it for transmission.
- **UART Transmitter:** The encrypted data is then sent out serially from Board 1 to Board 2 through the UART transmitter. This allows the encrypted information to be securely transmitted to the second board, maintaining the confidentiality of the data during transfer.

3. Board 2 Processing (Decryption)

- **UART Receiver:** Upon arrival at Board 2, the encrypted 8-bit data is received serially through its UART receiver. This receiver operates similarly to Board 1's, capturing the incoming data byte by byte.
- **Serial-In Parallel-Out (SIPO):** The encrypted data is then shifted into a 128-bit SIPO register on Board 2. This register collects 16 bytes of incoming data, corresponding to the full 128-bit AES-encrypted block.
- **AES Decryption:** Once the entire 128-bit block is received, the AES decryption module is invoked. Using the same 128-bit key that was employed for encryption on Board 1, the decryption module restores the original plaintext data.
- **Parallel-In Serial-Out (PISO):** The decrypted 128-bit data is subsequently loaded into a PISO register, which shifts the data out as an 8-bit serial stream, readying it for UART transmission.

- **UART Transmitter:** Finally, the decrypted data is transmitted back to the PC or an Arduino via the UART transmitter on Board 2, effectively completing the secure communication loop.

4. Data Reception by PC and Arduino

- **Arduino as a Listener:** In this setup, the Arduino acts as a "thief," capturing the data that is transmitted from the Basys 3 FPGA board. While it can monitor the data, it does not interfere with the secure communication process.
- **PC as a Monitoring Station:** The PC serves as a monitoring station, receiving the decrypted data for analysis. This setup allows users to verify the integrity and correctness of the data transmitted through the system.

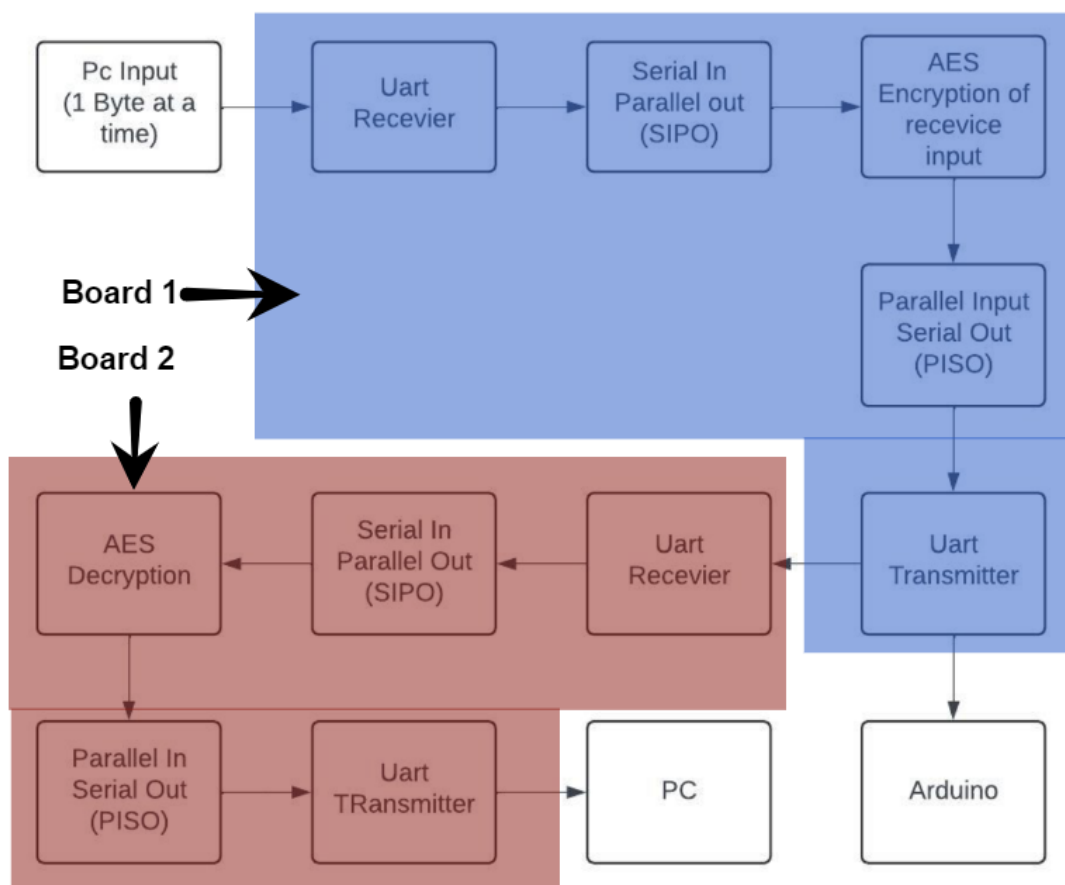


Figure 6.2: Data Transmission Flowchart

6.3 Hardware Setup

The hardware setup involves a Basys 3 FPGA board as the central hub for data communication. The setup ensures bidirectional data transfer between a PC, two Basys 3 boards, and an Arduino via UART communication. The hardware components are connected as follows:

6.3.1 PC to Basys 3 FPGA (Board 1)

- A PC running Tera Term communicates with the first Basys 3 FPGA board over a UART interface.
- The PC sends 8-bit data via the serial port, which is received by Basys 3 board 1.
- The data from the PC is transmitted via the UART interface to the first Basys 3 FPGA.

6.3.2 Basys 3 FPGA (Board 1) to Basys 3 FPGA (Board 2)

- Once the first Basys 3 FPGA receives the 8-bit data, it forwards this data via a UART interface to the second Basys 3 FPGA (Board 2).
- The second Basys 3 FPGA receives and processes the data for further communication.

6.3.3 Basys 3 FPGA (Board 1) to Arduino

- The same 8-bit data is also sent to an Arduino through a UART interface connected to Basys 3 board 1.
- The Arduino receives the data for processing or further action as per the program.

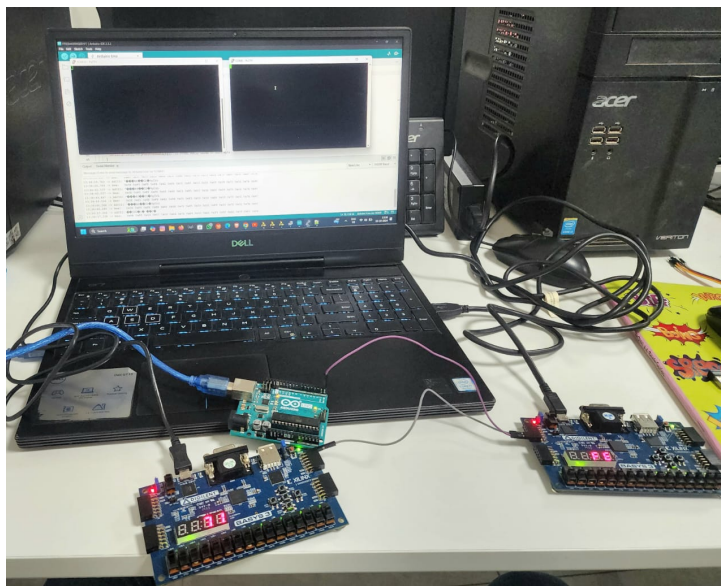


Figure 6.3: Hardware Setup

6.3.4 Basys 3 FPGA (Board 2) to PC

- Data is transmitted from the second Basys 3 FPGA (Board 2) back to the PC, where it is displayed on Tera Term.
- This allows the user to verify the successful transfer and reception of the data between the devices.

6.4 RTL Schematic

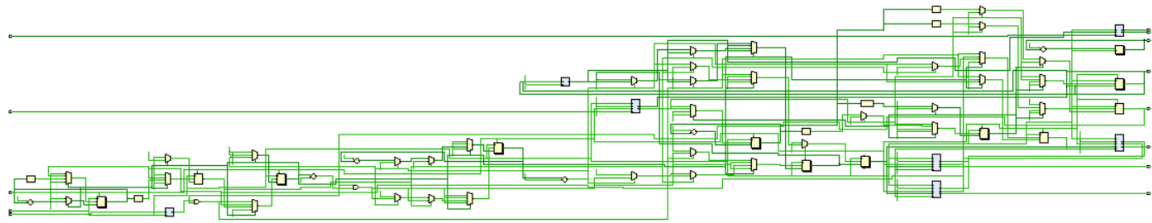


Figure 6.4: Hardware Setup

Chapter 7: Result

The AES-128 algorithm was successfully implemented on the Artix-7 FPGA for secure UART communication. Using Vivado 2014.4, the design was synthesized, tested, a bitstream file was generated, and encryption/decryption of 128-bit data was verified. Encrypted data transmitted via UART was securely received and decrypted on another FPGA board, confirming end-to-end functionality.

Performance testing revealed minimal encryption and decryption latency, with throughput comparable to standard UART communication. Resource utilization was efficient, with 36.01% of LUTs and 36.47% of Flip-Flops used, and robustness against data corruption was validated.

Hardware testing with a PC confirmed seamless UART protocol integration. Issues like timing violations and baud rate synchronization were addressed through pipeline optimization. This implementation highlights the viability of AES-128 for secure embedded communication, with potential for future enhancements such as support for higher key sizes and parallel processing to further enhance performance.

7.1 Safety Analysis and Improvements

Advanced Encryption Standard (AES) is widely used due to its robust security, efficiency, and adaptability across diverse platforms. Its applications span various industries, such as accounting information systems, where high-security assurance is critical. Conducting a safety analysis of AES is essential to evaluate its resistance against potential attacks and ensure its stability and usability.

7.1.1 Importance of Safety Analysis

Safety analysis evaluates an encryption algorithm's resilience, ensuring that attackers cannot compromise its security even with full knowledge of its structure. A robust algorithm requires attackers to expend significantly more resources and time on cryptanalysis than on brute force methods, thereby ensuring data protection.

7.1.2 Brute Force Attack

A brute force attack involves attempting all possible key combinations. For AES-128, the 128-bit key length necessitates 3.403×10^{38} combinations, which would take the fastest computers approximately 3.19×10^{14} years to crack. With an additional random 8-bit key, the combinations increase to 8.712×10^{40} , requiring around 8.165×10^{16} years to break. These figures demonstrate that brute force attacks are computationally infeasible, ensuring AES's time security.

No	Key size	Years Need
01	128 bit	3.19×10^{14} years
02	192 bit	5.88×10^{33} years
03	256 bit	1.0844×10^{53} years

Figure 7.1: Estimation of Years to Break AES

7.1.3 Square Attack

Square attacks exploit the byte-oriented structure of encryption algorithms and can affect reduced-round AES implementations. Research shows that square attacks succeed only on versions of AES with fewer than six rounds. To mitigate this, AES mandates at least 10 rounds for AES-128, ensuring its resistance to such attacks. This design choice reinforces the algorithm's robustness.

Chapter 8: Conclusion

This project successfully demonstrates the integration of the Advanced Encryption Standard (AES) for securing UART communication on an Artix-7 FPGA. By leveraging the robust security features of AES encryption, the system effectively addresses the inherent vulnerabilities of UART, ensuring data confidentiality and integrity during transmission.

The implementation process showcased a comprehensive approach to secure communication, where plaintext data is encrypted on the transmitting board and decrypted on the receiving board. The use of hardware-based AES not only enhances encryption speed and efficiency but also minimizes the potential for vulnerabilities that can arise from software-based implementations. The simulation results validated the correctness of the AES encryption and decryption processes, ensuring that the original data is accurately restored after transmission.

This work highlights the critical need for secure communication protocols in embedded systems and IoT applications, particularly as these systems increasingly interact with external networks. The proposed solution offers a reliable framework for secure data transfer, applicable in various contexts, from industrial control systems to sensitive data communication between microcontrollers.

Future work can explore optimizing the AES implementation further for resource-constrained devices and enhancing the system with additional security measures, such as authentication protocols or real-time monitoring mechanisms. Moreover, further improvements can focus on reducing the FPGA resource utilization while maintaining high throughput, ensuring scalability for broader applications. Overall, this project lays a solid foundation for advancing secure communication technologies in the rapidly evolving landscape of embedded systems.

References

- [1] Huang, Weilun, and Sheng, Guolun, Analysis and Research on UART Communication Protocol. In *2024 4th Asia-Pacific Conference on Communications Technology and Computer Science (ACCTCS)*, pp. 768–771. IEEE, 2024.
- [2] Kong, Lingxi, Niu, Qirui, and Yang, Pai, Design and Implementation of UART Based on Verilog HDL. In *Highlights in Science, Engineering and Technology*, vol. 38, pp. 949–955, 2023.
- [3] An, Dong, Guo, Zhouming, and Zhao, Kehan, Design of UART Communication System Based on Adaptive Baud Rate Technology. In *2023 International Conference on Design Science (ICDS)*, pp. 1–6. IEEE, 2023.
- [4] Rameel, Muhammad, and Asif, Zain, Fortifying Information Security: A Comparative Analysis of AES, DES, 3DES, RSA, and Blowfish Algorithm. In *Communications*, vol. 2, p. 5, 2024.
- [5] Hasija, Taniya, Kaur, Amanpreet, Ramkumar, KR, Sharma, Shagun, Mittal, Sudesh, and Singh, Bhupendra, A survey on performance analysis of different architectures of AES algorithm on FPGA. In *Modern Electronics Devices and Communication Systems: Select Proceedings of MEDCOM 2021*, pp. 39–54. Springer, 2023.
- [6] Priyanka Brahmaiah, V, Jaswanth, PV, Sri Likhitha, D, and Pallavi Sudha, M, Implementation of AES Algorithm. In *International Conference on Information and Communication Technology for Intelligent Systems*, pp. 161–171. Springer, 2023.
- [7] Lu, Zhengyi, Analysis on AES encryption standard and safety. In *Third International Symposium on Computer Engineering and Intelligent Communications (ISCEIC 2022)*, vol. 12462, pp. 292–297. SPIE, 2023.
- [8] Swayamprakash, Busam, Panditi Jeevana Shreshta, Chittapu Maheshwar Reddy, Jamal, K, Kiran Mannem, and M Suneetha, Design of Advanced Encryption Standard using Verilog HDL. In *2023 7th International Conference on Trends in Electronics and Informatics (ICOEI)*, pp. 676–682. IEEE, 2023.
- [9] Kinshuk, Musarrat Afreen, Mohd Asif Javed, Md Faizan Ahmad, and Rajiv Sharma, Promoting Sustainable Security by Implementing AES Algorithm for Enhanced Confidentiality in UART Serial Communication. In *International Conference on Sustainable Development through Machine Learning, AI and IoT*, pp. 232–243. Springer, 2024.
- [10] Sharmila, S, Bhuvaneswaran, RS, Natarajan, Kailash, Prathiba, A, and Vaithiyanathan, D, Hardware Implementation of Block Ciphers—A Case Study on Encrypted Image Transfer Over Universal Asynchronous Receiver Transmitter. In *2023 International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS)*, pp. 1120–1128. IEEE, 2023.
- [11] Cupi, Pasquale, Evaluation and Enhancement of Security in Serial Communication for Mechatronic Systems. PhD Thesis, Politecnico di Torino, 2024.